

1. Shell Basics

What is a Shell?

A **shell** is a command-line interface that allows users to interact with the operating system. It interprets the commands you type and communicates them to the kernel. Common shells include:

- **Bash (Bourne Again SHell)**: Default on most Linux distributions, widely used, supports scripting.
- **Zsh (Z Shell)**: Similar to Bash but with more features like enhanced tab completion and themes.
- **Dash (Debian Almquist Shell)**: Lightweight shell, faster than Bash, often used in system scripts.

For learning shell scripting, we will focus on **Bash**, as it is the most commonly used and versatile.

Running Commands and Scripts

You can run commands directly in the terminal, or you can write them in a **script file** (`.sh`) for repeated use.

Example of a simple command:

```
echo "Hello World"
```

Writing a Shell Script

A shell script is a plain text file containing a series of commands. Example:

```
myscript.sh
```

```
#!/bin/bash
echo "Hello World"
```

- The first line, `#!/bin/bash`, is called a **shebang**. It tells the system which interpreter to use to run the script.
- Lines starting with `#` are **comments** and are ignored during execution.

Making a Script Executable

Before running a script, you need to give it execute permission:

```
chmod +x myscript.sh
```

This allows the script to be executed as a program.

Running the Script

After giving execute permission, run the script using:

```
./myscript.sh
```

Output:

```
Hello World
```

Note: Using `./` tells the shell to look for the script in the current directory.

Example: Simple Script with Variables

```
#!/bin/bash

name="Farhan"
echo "Hello, $name!"
```

Output:
Hello, Farhan!

2. Variables and Data

Declaring Variables

In Bash, you can declare variables in two main ways:

- 1. **Local variable (only for current shell/script):**
var=value
- 2. **Environment variable (exported to child processes):**
export var=value

Example:

```
name="Farhan"
echo "My name is $name"
```

Accessing Variables

- **\$var** – Access the value of the variable.
- **\${var}** – Use braces to avoid ambiguity, especially when appending text.

Example:

```
name="Farhan"
echo ${name}man # Output: Farhanman
```

Variable	Meaning
\$0	Name of the script
\$1, \$2...\$n	Command-line arguments
\$#	Number of arguments
\$@	All arguments as separate words
\$?	Exit status of the last command
\$\$	Process ID (PID) of the script

Example:

```
echo "Script name: $0"
echo "First arg: $1"
echo "Number of args: $#"
```

```
echo "All args: $@"
echo "Exit code of last cmd: $?"
echo "PID of this script: $$"
```

Arithmetic Operations

Use `$((expression))` for calculations:

```
var=5
echo $var # Output: 5

a=3; b=2
echo $((a+b)) # Output: 5
echo $((a*b)) # Output: 6
```

String Operations

- `${#var}` – Length of a string
- `${var:position:length}` – Substring

Example:

```
name="Farhan"
echo ${#name} # Output: 6 (length of string)
echo ${name:0:3} # Output: Far (first 3 characters)
```

3. Floating Point Evaluation Using `bc -l`

Why `bc -l`?

Bash only supports **integer arithmetic** natively. To perform **floating-point calculations**, you need an external tool like `bc` (Basic Calculator).

- `bc -l` enables **floating-point arithmetic** and provides standard math functions like `sine`, `cosine`, `sqrt`, `log`, etc.

Basic Usage

You can pass an expression to `bc -l` using `echo` and a pipe:

Example:

```
echo "3.5 + 2.2" | bc -l
```

Output:

```
5.7
```

Assigning Result to a Variable

You can store the result of a calculation in a variable using **command substitution**:

Example:

```
result=$(echo "3.5 + 2.2" | bc -l)
echo "Result: $result"
```

Output:

```
Result: 5.7
```

More Examples

1. Subtraction and Multiplication

```
echo "10.5 - 4.2" | bc -l # Output: 6.3
echo "3.5 * 2.0" | bc -l # Output: 7.0
```

2. Division

```
echo "5 / 2" | bc -l # Output: 2.5000000000
```

3. Using Variables

```
a=3.5
b=2.2
result=$(echo "$a + $b" | bc -l)
echo "Sum: $result" # Output: Sum: 5.7
```

Math Functions with `-l`

`bc -l` provides standard math functions:

- **sqrt(x)** – Square root
- **s(x)** – Sine (x in radians)
- **c(x)** – Cosine (x in radians)
- **l(x)** – Natural logarithm

Example:

```
echo "scale=4; sqrt(10)" | bc -l # scale sets decimal precision →
Output: 3.1622
echo "scale=4; s(3.14159/2)" | bc -l # Output: 1.0000
```

Setting Decimal Precision

`scale` controls the number of digits after the decimal point:

Example:

```
echo "scale=3; 10 / 3" | bc -l
```

Output:

```
3.333
```

`scale` must be set **inside the expression**, otherwise `bc` defaults to 0 decimal places.

4. Input / Output

Printing Output

1. **echo** – The simplest way to print text to the terminal:

```
echo "Hello World"
```

You can also print variables:

```
name="Farhan"
echo "My name is $name"
```

2. **printf** – More control over formatting (similar to C's printf):

```
printf "Name: %s, Age: %d\n" "Farhan" 25
```

- `%s` is for string, `%d` is for integers
- `\n` is for a new line

Reading Input from User

read allows you to take input from the user:

Basic usage:

```
read variable_name
```

Example:

```
read name
echo "Hello, $name!"
```

Prompting the user while reading:

```
read -p "Enter your age: " age
echo "Your age is $age"
```

Silent input (useful for passwords):

```
read -s password
echo "Password entered!"
```

You can store the output of a command in a variable using:

Dollar parentheses (preferred):

```
var=$(date)
```

Example:

```
current_dir=$(pwd)
echo "You are in directory: $current_dir"
```

Summary of Options for **read**

- **-p** : Prompt text before reading input
- **-s** : Silent mode (does not display input on screen)
- **-n N** : Read only N characters
- **-t N** : Timeout after N seconds

5. Control Structures

Conditional Execution

Bash allows you to execute commands **based on conditions**:

- **command1 && command2** – Run **command2** only if **command1** succeeds (exit status 0)
- **command1 || command2** – Run **command2** only if **command1** fails (non-zero exit status)
- **command1 ; command2** – Run **command1** then **command2** regardless of success

Example:

```
mkdir mydir && echo "Directory created"
mkdir mydir || echo "Directory already exists"
echo "Hello"; echo "World"
```

if ... then ... elif ... else ... fi

Bash uses **if statements** to execute commands conditionally:

Syntax:

```
if [ condition ]; then
    commands
elif [ condition ]; then
    commands
else
    commands
fi
```

Example:

```
if [ $1 -gt 10 ]; then
    echo "Greater than 10"
elif [ $1 -eq 10 ]; then
    echo "Equal to 10"
else
    echo "Less than 10"
fi
```

Test Expressions

Using `[ ]` or `[[ ]]` or **test**

String Comparison:

- `=` or `==` : Equal
- `!=` : Not equal
- `<` : Less than (alphabetical)
- `>` : Greater than (alphabetical)
- `-z string` : True if string is empty
- `-n string` : True if string is not empty

Integer Comparison:

- `-eq` : Equal
- `-ne` : Not equal
- `-gt` : Greater than
- `-ge` : Greater than or equal
- `-lt` : Less than
- `-le` : Less than or equal

File Tests:

- `-a file` : File exists
- `-d file` : Directory exists
- `-f file` : Regular file exists
- `-r file` : Readable
- `-w file` : Writable
- `-x file` : Executable

Logical Operators (within [] or [[]])

- `&&` : Logical AND (works with `[[ ]]`)
- `||` : Logical OR (works with `[[ ]]`)
- `!` : Logical NOT
- `-a` : Logical AND (works with `[ ]`)
- `-o` : Logical OR (works with `[ ]`)

Examples:

Using `[[ ]]` with `&&` and `||`:

```
if [[ -f "$1" && -r "$1" ]]; then
    echo "File exists and is readable"
fi
```

Using `[ ]` with `-a` and `-o`:

```
if [ -f "$1" -a -r "$1" ]; then
    echo "File exists and is readable"
fi
```

```
if [ -f "$1" -o -d "$1" ]; then
    echo "It is either a file or a directory"
fi
```

Case Statement

case ... esac allows checking a variable against multiple patterns:

Syntax:

```
case variable in
    pattern1) commands ;;
    pattern2) commands ;;
    *) default commands ;;
esac
```

Example:

```
case $1 in
    start) echo "Starting service" ;;
    stop) echo "Stopping service" ;;
    *) echo "Unknown option" ;;
esac
```

6. Loops

For Loops

1. **Iterating over a list:**

```
for X in cyan magenta yellow
do
    echo $X
done
```

Output:

```
cyan
```

```
magenta
yellow
```

2. Iterating over a range of numbers (Bash 4.0+):

```
for i in {1..5}; do
    echo "Number: $i"
done
```

Output:

```
Number: 1
Number: 2
Number: 3
Number: 4
Number: 5
```

3. C-style for loop:

```
for (( i=1; i<=5; i++ )); do
    echo "Number: $i"
done
```

While Loops

Basic while loop:

```
i=1
while [ $i -le 10 ]; do
    echo "$i"
    i=$((i+1))
done
```

Reading a file line by line:

```
filename="input.txt"
IFS=
while read -r line
do
    echo "Line: $line"
done < "$filename"
```

Explanation:

- `read -r line` reads one line at a time into the variable `line`
- `-r` prevents backslashes (`\`) from being treated as escape characters
- `IFS=` clears the Internal Field Separator so that leading/trailing spaces are preserved

Until Loops

An **until loop** executes commands **until a condition becomes true** (opposite of while):

```
i=1
until [ $i -gt 5 ]; do
    echo "Number: $i"
    i=$((i+1))
done
```


Output:

```
Number: 1
Number: 2
Number: 3
Number: 4
Number: 5
```

Loop Control

- **break** : Exit the loop immediately
- **continue** : Skip the rest of the current iteration and continue with the next

Example:

```
for i in {1..5}; do
    if [ $i -eq 3 ]; then
        continue
    fi
    echo $i
done
```

Output:

```
1
2
4
5
```

7. Functions

Defining Functions

In Bash, a function is a reusable block of code that can be called multiple times.

Syntax:

```
myfunc() {
    commands
}
```

Example:

```
greet() {
    echo "Hello, $1!"
}
```

greet Farhan

Output:

```
Hello, Farhan!
```

Passing Arguments to Functions

Arguments passed to a function are accessed using **\$1**, **\$2**, ... **\$n**:

Example:

```
sum() {
    echo $(( $1 + $2 ))
}
```

```
sum 1 2
```

Output:

```
3
```

Returning Values

Bash functions can “return” values in two ways:

1. Using **echo** and capturing the output (most common):

```
add() {  
    local sum=$(( $1 + $2 ))  
    echo $sum  
}
```

```
result=$(add 5 3)  
echo "The sum is $result"
```

Output:

```
The sum is 8
```

- `local` keyword restricts the variable to the function scope

2. Using **return** – can only return integers between 0–255 (typically used for exit codes, not calculations):

```
check_even() {  
    if [ $(( $1 % 2 )) -eq 0 ]; then  
        return 0  
    else  
        return 1  
    fi  
}
```

```
check_even 4  
if [ $? -eq 0 ]; then  
    echo "Number is even"  
else  
    echo "Number is odd"  
fi
```

Output:

```
Number is even
```

8. Redirection and Pipes

File Descriptors

In Bash, commands can produce **normal output** and **error messages**. These are sent to different **file descriptors**:

- **0** – stdin → where a command reads input from (usually the keyboard).
- **1** – stdout → where a command writes normal output (usually the terminal).
- **2** – stderr → where a command writes error messages (usually the terminal).

Redirecting Output

1. Overwrite a file (`>`):
`echo "Hello, World!" > output.txt`
2. Append to a file (`>>`):
`echo "Another line" >> output.txt`

Redirecting Error Messages

1. Redirect **stderr** to a file (`2>`)

You can redirect error messages to a separate file so they don't mix with normal output:

Example:

```
ls /nonexistent 2> error.log
```

- Here, `2>` tells the shell to send the **stderr** (file descriptor 2) to `error.log`.
- Normal output (stdout) will still appear on the terminal.

2. Redirect both **stdout** and **stderr** to the same file (`> file 2>&1`)

Sometimes you want **all output**, both normal and error, in one file:

Example:

```
ls /tmp /nonexistent > all_output.log 2>&1
```

- `> all_output.log` redirects **stdout** to the file.
- `2>&1` tells the shell to redirect **stderr (2)** to the **same place as stdout (1)**.

Common mistake: `2>1` is **not correct**, because without `&`, the shell interprets `1` as a **filename**, not a file descriptor.

- `&` indicates that what follows is a **file descriptor**.
- `>&` is sometimes called a **redirect merger operator** because it merges stdout and stderr into the same stream.

Why Use This?

- Keeps logs organized by separating errors from normal output.
- Useful in scripts where you want to handle errors differently from regular output.
- Makes debugging easier by capturing error messages in a separate file.

Sending Output to **stderr**

Sometimes, scripts need to explicitly send messages to **stderr**:

Example:

```
#!/bin/bash
```

```
do_something() {  
    if [ ! -f "important_file.txt" ]; then  
        echo "Error: important_file.txt not found!" >&2  
        return 1 # non-zero indicates failure  
    fi  
}
```

```

        echo "File exists!"
    }

do_something

# Check if an error occurred
if [ $? -ne 0 ]; then
    echo "Script encountered an error" >&2
    exit 1 # exit with non-zero status
fi

```

- `>&2` sends output explicitly to **stderr**.
- `?` stores the exit status of the last command. Non-zero usually indicates an error.

Exercises:

Exercise-1: User Authentication Script

Write a shell script that:

1. Prompts the user to enter a **username**.
2. Prompts the user to enter a **password** (the input must be hidden using `read -s`).
3. Checks a file named `userdata.txt` which contains **username–password pairs**, separated by a space.
Example contents of `userdata.txt`:

```
alice 12345
bob qwerty
charlie pass123
```
4. If the entered username and password pair **matches an entry** in the file → display a welcome message.
5. If it does not match → display an error message.

Exercise-2: Fun Todo List Manager

Write a shell script `todo.sh` that supports the following commands:

1. Add a task

```
./todo.sh add "Read Bash notes"
```

Appends the task to `todo.txt` as:

```
[ ] Read Bash notes
```
2. List tasks

```
./todo.sh list
```

Prints all tasks with numbering. Example–

```
1. [ ] Read Bash notes
2. [ ] Finish OS lab
3. [x] Submit assignment
```
3. Mark task as done

```
./todo.sh done 2
```

Marks the 2nd task as done by replacing `[ ]` with `[x]` and prints a motivational message

```
Nice job 🚀
```

4. Clear all tasks

```
./todo.sh clear
```

Empties the file `todo.txt` and prints:

```
Todo list cleared 🗑️
```

5. Search tasks by keyword

```
./todo.sh search "lab"
```

Prints only tasks containing `"lab"`